Phala Network TEE 구현체 분석 - 성능 및 최적화

1. 성능 특성 분석

1.1 TEE 환경 성능 제약

```
SGX Enclave 제약사항:
├── 메모리 제한: 최대 256MB (일반적으로 128MB)
├── CPU 오버헤드: 5-15% 성능 저하
├── I/O 제약: 모든 I/O가 호스트를 경유
├── 컨텍스트 스위칭: Enclave 진입/탈출 비용
└── 메모리 암호화: 하드웨어 암호화 오버헤드
```

1.2 컴포넌트별 성능 프로파일

```
# pRuntime 성능 특성
├── 스레드 구성 (threads-in-pruntime.md 기반)
| ├── main thread: 1개 (제어)
| ├── benchmark: --cores개 (성능 측정)
| ├── rayon pool: CPU cores개 (병렬 컴파일)
| ├── web pool: 설정값 (HTTP API)
| ├── sidevm pool: --cores + 2개 (VM 실행)
```

```
| └── tokio blocking: 최대 16개 (비동기-동기 변환)
|
├── Gramine 추가 오버헤드
| ├── IPC: 1개 스레드
| ├── Async events: 1개 스레드
| └── TLS-handshaking: 1개 스레드
```

2. 메모리 최적화

2.1 phala-allocator 최적화

```
// 메모리 할당 최적화 전략 (추정)
struct PhalaAllocator {
// 메모리 풀 관리
small_pools: Vec<MemoryPool>, // 작은 객체용
large_pool: LargeObjectPool, // 큰 객체용
// 캐시 친화적 할당
cache_aligned_pool: AlignedPool, // 캐시 라인 정렬
// 가비지 컬렉션
gc_threshold: usize, // GC 트리거 임계값
fragmentation_limit: f32, // 단편화 한계
}
```

```
impl PhalaAllocator {
fn  allocate_optimized(&mut  self, size: usize, align: usize) -> *mut u8 {
match  size {
0..=64 => self.small_pools[0].allocate(size, align),
65..=256 => self.small_pools[1].allocate(size, align),
257..=1024 => self.small_pools[2].allocate(size, align),
_ => self.large_pool.allocate(size, align),
}
}
}
```

2.2 상태 저장 최적화

```
// Trie 스토리지 최적화 (phala-trie-storage)
struct OptimizedTrieStorage {
// 압축된 노드 저장
compressed_nodes: CompressedNodeCache,
// 자주 접근하는 경로 캐싱
hot_path_cache: LruCache<Vec<u8>, TrieNode>,
// 배치 업데이트
pending_updates: BatchUpdateBuffer,
// 메모리 매핑
mmap_storage: Option<MmapFile>,
```

```
}
```

3. CPU 최적화

3.1 WASM 실행 최적화

```
// Pink Runtime WASM 최적화
struct OptimizedWasmEngine {
// JIT 컴파일 캐시
compiled_cache: HashMap<CodeHash, CompiledModule>,
// 인터프리터 최적화
interpreter_config: InterpreterConfig {
inline_threshold: 100,
unroll_loops: true,
optimize_calls: true,
},
// 메모리 사전 할당
preallocated_memory: Vec<WasmMemoryPool>,
}

impl OptimizedWasmEngine {
fn execute_with_cache(&mut self, code_hash: &CodeHash, input: &[u8]) -> Result
let module = match self.compiled_cache.get(code_hash) {
```

```
Some(cached) => cached,

None => {

let  compiled = self.compile_and_optimize(code_hash)?;

self.compiled_cache.insert(*code_hash, compiled);

self.compiled_cache.get(code_hash).unwrap()

}

};

module.execute_optimized(input)

}

}
```

3.2 암호화 연산 최적화

```
// phala-crypto 최적화

struct CryptoOptimizer {

// 하드웨어 가속 활용

aes_ni_available: bool,

sha_extensions: bool,

// 배치 처리

batch_processor: BatchCryptoProcessor,

// 키 캐싱

key_cache: LruCache<KeyId, CachedKey>,

}
```

```
impl CryptoOptimizer {
fn batch_sign(&mut self, messages: &[&[u8]], key: &PrivateKey) -> Vec<Signatur
// 배치로 서명 생성하여 오버헤드 감소
self.batch_processor.sign_batch(messages, key)
}
fn hardware_accelerated_hash(&self, data: &[u8]) -> Hash {
if self.sha_extensions {
// SHA 하드웨어 확장 사용
hardware_sha256(data)
} else {
// 소프트웨어 구현
software_sha256(data)
}
}
}
```

4. 네트워크 최적화

4.1 HTTP API 최적화 (Rocket)

```
// pRuntime HTTP 서버 최적화
#[derive(Serialize, Deserialize)]
```

```rust
struct OptimizedApiConfig {
// 연결 풀링
keep_alive: bool,
max_connections: usize,
connection_timeout: Duration,
// 압축
enable_compression: bool,
compression_level: u32,
// 캐싱
response_cache_size: usize,
cache_ttl: Duration,
}

#[rocket::async_trait]
impl Fairing for PerformanceOptimizer {
async fn on_request(&self, req: &mut Request<'_>, _: &mut Data<'_>) {
// 요청 압축 해제
if req.headers().get_one("Content-Encoding") == Some("gzip") {
// 압축 해제 로직
}
// 캐시 확인
if let Some(cached) = self.check_cache(req.uri().path()) {
// 캐시된 응답 반환
```

```
}
}
async fn on_response<'r>(&self, req: &'r Request<'_>, res: &mut Response<'r>)
// 응답 압축
if self.should_compress(req, res) {
// 압축 로직
}
// 캐시 저장
self.cache_response(req.uri().path(), res);
}
}
```

4.2 Pherry 통신 최적화

```
// Pherry 배치 처리 최적화
struct BatchProcessor {
batch_size: usize,
batch_timeout: Duration,
pending_requests: Vec<SyncRequest>,
}

impl BatchProcessor {
async fn process_batch(&mut self) -> Result<()> {
```

```
if self.pending_requests.len() >= self.batch_size
|| self.batch_timeout_reached() {
let batch = std::mem::take(&mut self.pending_requests);
// 배치 요청 생성
let batch_request = BatchSyncRequest {
headers: batch.into_iter().map(|r| r.header).collect(),
};
// 단일 HTTP 요청으로 배치 처리
self.tee_client.batch_sync(batch_request).await?;
}
Ok(())
}
}
```

5. 스토리지 최적화

5.1 상태 압축

```
// 상태 데이터 압축
struct StateCompressor {
compression_algorithm: CompressionAlgorithm,
compression_level: u32,
dictionary: Option<CompressionDictionary>,
```

```
}

impl StateCompressor {
fn  compress_state(&self, state: &StateData) -> CompressedState {
match  self.compression_algorithm {
CompressionAlgorithm::Lz4 => {
// 빠른 압축/해제를 위한 LZ4
lz4_compress(state, self.compression_level)
}
CompressionAlgorithm::Zstd => {
// 높은 압축률을 위한 Zstandard
zstd_compress_with_dict(state, &self.dictionary)
}
}
}
}
```

5.2 캐시 계층화

```
// 다층 캐시 시스템
struct LayeredCache {
l1_cache: LruCache<StateKey, StateValue>, // 메모리 캐시
l2_cache: Option<SsdCache>, // SSD 캐시
```

```
l3_cache: PersistentStorage, // 영구 저장소
}

impl LayeredCache {
async fn get_optimized(&mut self, key: &StateKey) -> Option<StateValue> {
// L1 캐시 확인
if let Some(value) = self.l1_cache.get(key) {
return Some(value.clone());
}
// L2 캐시 확인
if let Some(ref mut l2) = self.l2_cache {
if let Some(value) = l2.get(key).await {
self.l1_cache.put(key.clone(), value.clone());
return Some(value);
}
}
// L3 스토리지 확인
if let Some(value) = self.l3_cache.get(key).await {
self.l1_cache.put(key.clone(), value.clone());
if let Some(ref mut l2) = self.l2_cache {
l2.put(key.clone(), value.clone()).await;
}
return Some(value);
```

```
}

None

}

}
```

6. 동시성 최적화

6.1 비동기 처리 최적화

```
// Tokio 런타임 최적화

struct OptimizedRuntime {

runtime: tokio::runtime::Runtime,

scheduler_config: SchedulerConfig,

}


impl OptimizedRuntime {

fn  new_optimized() -> Self {

let  runtime = tokio::runtime::Builder::new_multi_thread()

.worker_threads(num_cpus::get())

.max_blocking_threads(16)

.thread_stack_size(2 * 1024 * 1024) // 2MB 스택

.thread_name("phala-worker")

.enable_all()
```

```
.build()
.expect("Failed to create runtime");
Self {
runtime,
scheduler_config: SchedulerConfig::default(),
}
}
}
```

6.2 락 최적화

```
// 락 프리 데이터 구조 활용
use crossbeam::queue::ArrayQueue;
use parking_lot::RwLock; // 더 빠른 RwLock

struct LockOptimizedStorage {
// 읽기 전용 데이터는 Arc로 공유
immutable_data: Arc<ImmutableState>,
// 자주 변경되는 데이터는 락 프리 큐 사용
update_queue: ArrayQueue<StateUpdate>,
// 필요시에만 락 사용 (parking_lot 사용)
mutable_state: RwLock<MutableState>,
```

```
}
```

7. 벤치마킹 및 프로파일링

7.1 성능 측정

```
// 벤치마크 스레드 활용
struct PerformanceBenchmark {
benchmark_threads: usize,
metrics_collector: MetricsCollector,
}

impl PerformanceBenchmark {
fn  run_benchmark(&self, workload: Workload) -> BenchmarkResult {
let  start = Instant::now();
// 벤치마크 스레드에서 워크로드 실행
let  results = (0..self.benchmark_threads)
.into_par_iter()
.map(|_| self.execute_workload(&workload))
.collect::<Vec<_>>();
let  duration = start.elapsed();
BenchmarkResult {
throughput: results.len() as f64 / duration.as_secs_f64(),
```

```
latency_p50: calculate_percentile(&results, 0.5),
latency_p95: calculate_percentile(&results, 0.95),
latency_p99: calculate_percentile(&results, 0.99),
}
}
}
```

7.2 실시간 모니터링

```
// 성능 메트릭 수집
struct PerformanceMonitor {
cpu_usage: AtomicU64,
memory_usage: AtomicU64,
request_count: AtomicU64,
error_count: AtomicU64,
response_times: RwLock<VecDeque<Duration>>,
}

impl PerformanceMonitor {
fn  collect_metrics(&self) -> PerformanceMetrics {
PerformanceMetrics {
cpu_usage_percent: self.cpu_usage.load(Ordering::Relaxed) as f64 / 100.0,
memory_usage_mb: self.memory_usage.load(Ordering::Relaxed) / 1024 / 1024,
```

```
requests_per_second: self.calculate_rps(),

error_rate: self.calculate_error_rate(),

avg_response_time: self.calculate_avg_response_time(),

}

}

}
```

8. 최적화 권장사항

8.1 배포 최적화

```
# Cargo.toml 프로덕션 설정

[profile.production]

inherits = "release"

lto = true # 링크 타임 최적화

codegen-units = 1 # 단일 코드 생성 유닛

panic = "abort" # 패닉 시 즉시 종료

opt-level = 3 # 최대 최적화

debug = false # 디버그 정보 제거
```

8.2 런타임 튜닝

```
# 환경 변수 최적화

export  RUST_LOG="warn"  # 로그 레벨 최소화
```

```
export MALLOC_ARENA_MAX=2  # 메모리 아레나 제한
export MALLOC_MMAP_THRESHOLD_=131072  # mmap 임계값
export RUST_BACKTRACE=0  # 백트레이스 비활성화
```

8.3 하드웨어 최적화

- **CPU**: Intel SGX 지원 CPU (최신 세대 권장)
- **메모리**: 충분한 RAM (최소 16GB, 권장 32GB+)
- **스토리지**: NVMe SSD (높은 IOPS 필요)
- **네트워크**: 낮은 지연시간 네트워크 연결

이 문서는 Phala Network TEE 구현체의 성능 특성과 최적화 방안을 다룹니다. 마지막 문서에서는 보안 고려사항과 모범 사례를 분석합니다.